IC526 Advanced Compilers

Lec 17: mem2reg

Yoonseung Kim
DGIST EECS
Fall 2026

Announcement: Individual Project

- The [lib/irgen.ml] must be in your assignment repository.
- Check out the specification document.
- Presentations slots will be open in the GitHub Issue page.

Last Time: A Transformation by Hand

Memory-based IR

```
E:
  %p = alloca i32
  store i32 1, ptr %p
  br i1 %c, label %T, label %X
T:
  store i32 2, ptr %p
  br label %X
X:
  %r = load i32, ptr %p
  ret i32 %r
```

Promoted IR

```
E:
  br i1 %c, label %T, label %X
T:
  br label %X
X:
  %r = phi i32 [1,%E],[2,%T]
  ret i32 %r
```

We will promote memory slots to regsiters through a sequence of IR transformations.

Introduction to Register Promotion

A Slot in a Single Block

Before

```
%p = alloca i32
store i32 %a, ptr %p
%old = load i32, ptr %p
%next = add i32 %old, 1
store i32 %next, ptr %p
%r = load i32, ptr %p
ret i32 %r
```

After

```
%next = add i32 %a, 1
ret i32 %next
```

Which Slots Can mem2reg Promote?

Direct loads and stores

```
%p = alloca i32
store i32 0, ptr %p
%r = load i32, ptr %p
ret i32 %r
```

The slot can disappear.

Address passed to an unknown function

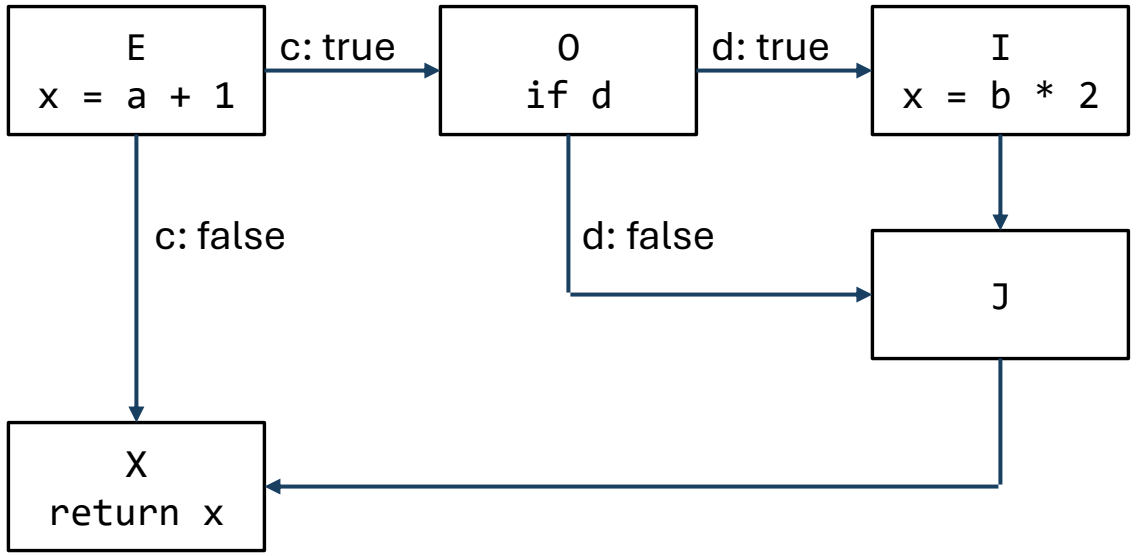
```
%p = alloca i32
store i32 0, ptr %p
call void @touch(ptr %p)
%r = load i32, ptr %p
ret i32 %r
```

The call may change the slot.

LLVM mem2reg checks allocas in the entry block.

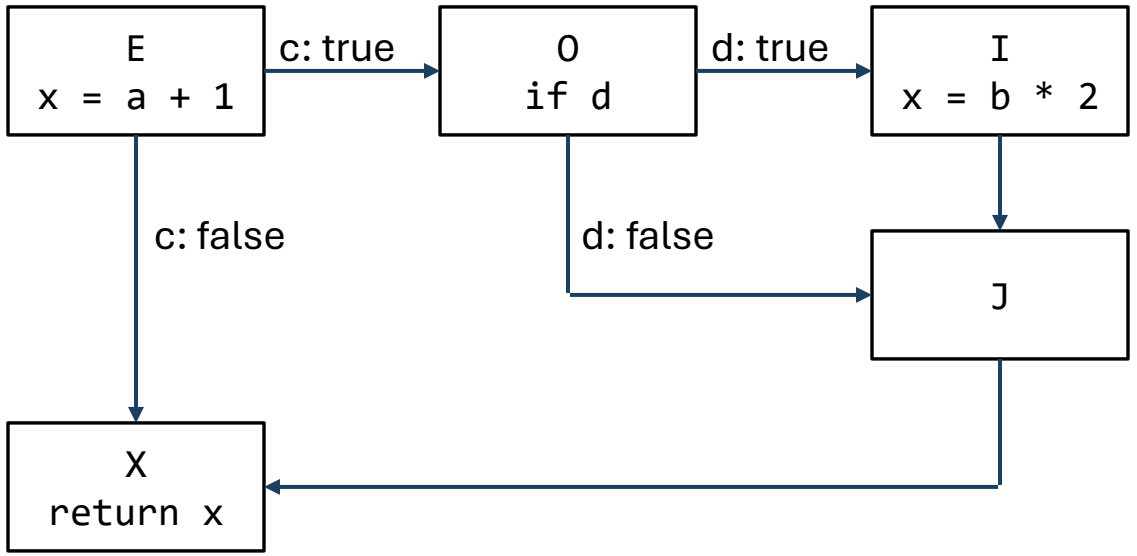
Running Example: Nested Conditionals

```
fn nested(a: i32, b: i32,
          c: bool, d: bool) -> i32 {
  let mut x: i32 = a + 1;
  if c {
    if d { x = b * 2; }
  }
  return x;
}
```



Nested Conditionals: Initial IR

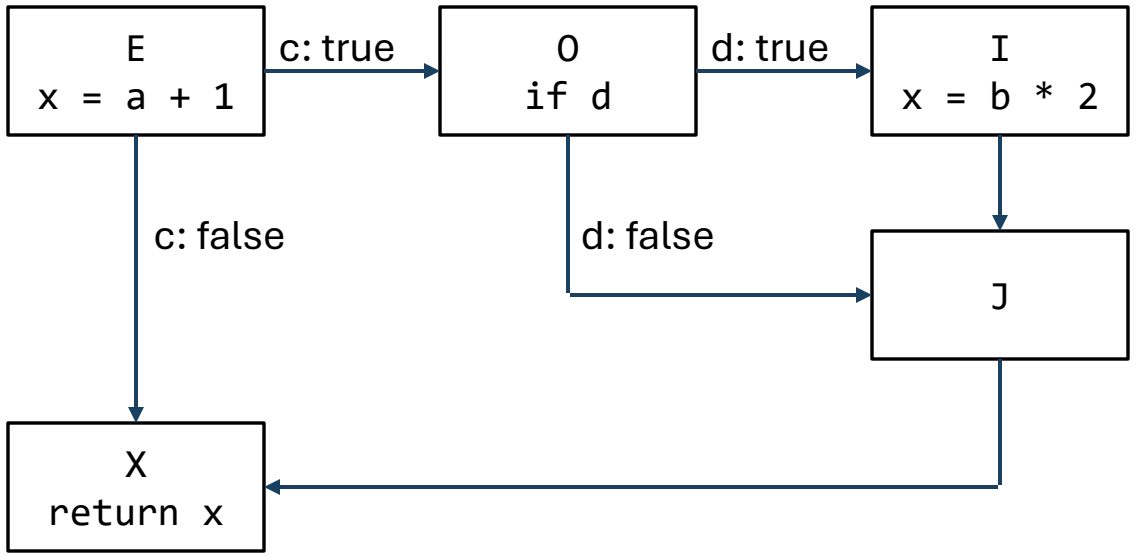
```
E:
  %p = alloca i32
  %initial = add i32 %a, 1
  store i32 %initial, ptr %p
  br i1 %c, label %O, label %X
O:
  br i1 %d, label %I, label %J
I:
  %updated = mul i32 %b, 2
  store i32 %updated, ptr %p
  br label %J
J:
  br label %X
X:
  %r = load i32, ptr %p
  ret i32 %r
```



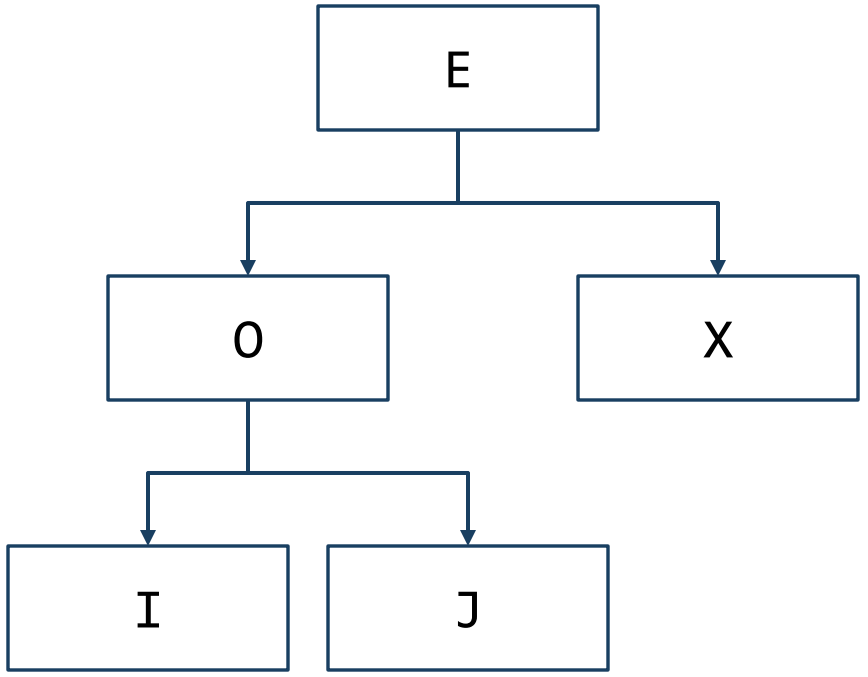
Dominator Trees

The Dominator Tree

Control-flow graph



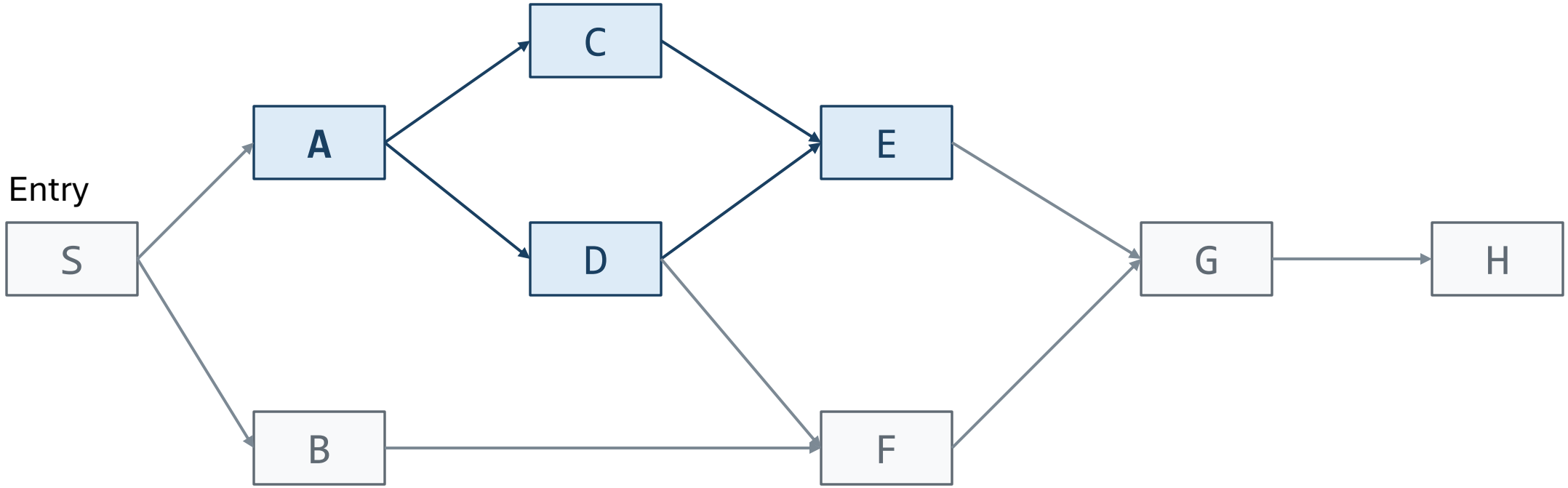
Dominator tree



Advanced Topic: Why tree?

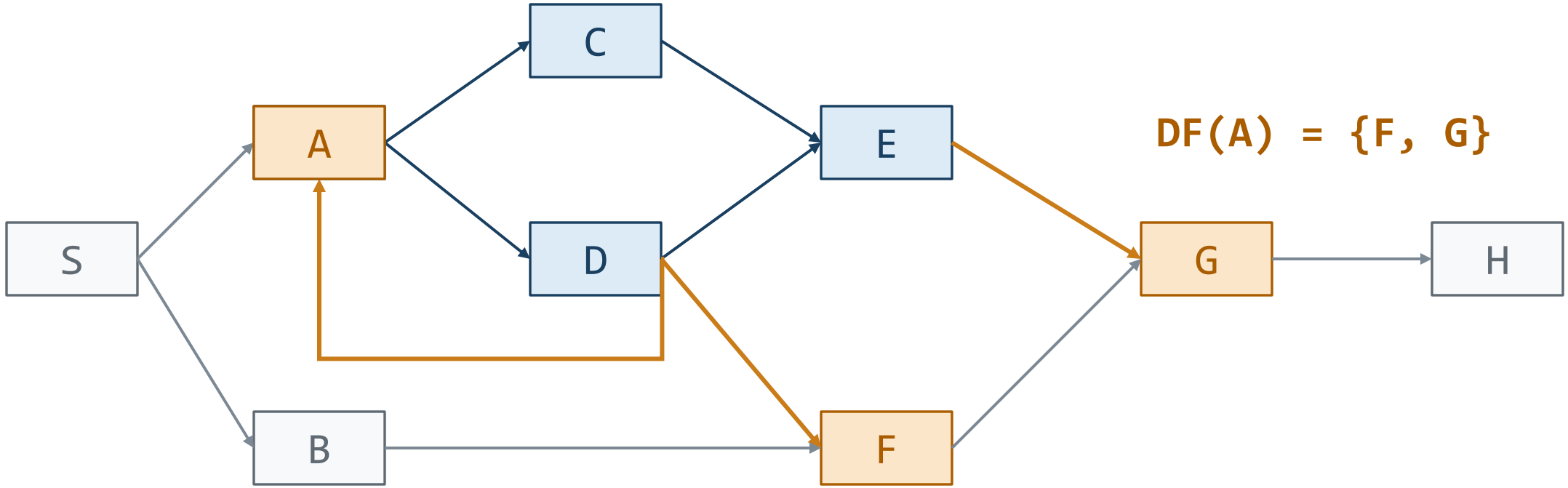
Dominance Frontier

A dominates {A, C, D, E}



Dominance Frontier

Y ∈ DF(A) when A dominates a predecessor of Y,
but A does not strictly dominate Y.



mem2reg

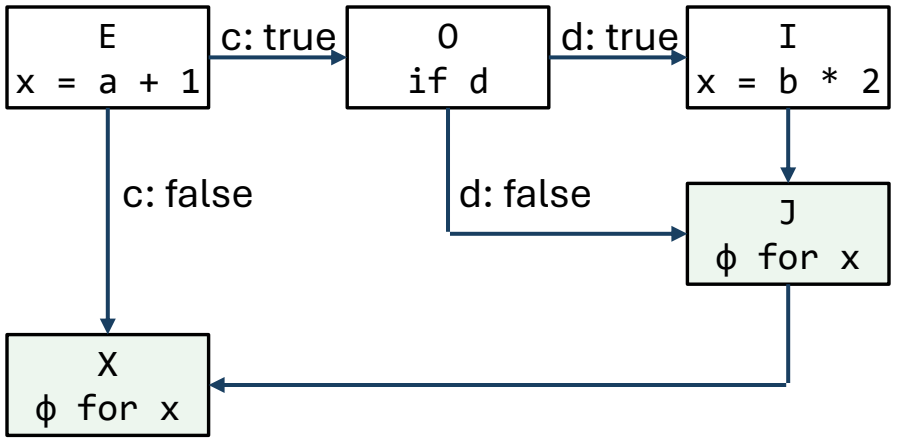
Steps To Promote Registers

- Step 1: Place (incomplete) ϕs at the DF of the blocks that store to the slot.
- Step 2: Complete ϕs using the dominator tree
- Step 3: Replace load values with ϕ values
- Step 4: Remove stores & alloca that are not used anymore

Nested 1: Mark the ϕ Locations

```
E:
  %p = alloca i32
  %initial = add i32 %a, 1
  store i32 %initial, ptr %p
  br i1 %c, label %O, label %X
O:
  br i1 %d, label %I, label %J
I:
  %updated = mul i32 %b, 2
  store i32 %updated, ptr %p
  br label %J
J:
  %x.j = phi i32 [?,%O],[?,%I]
  br label %X
X:
  %x.x = phi i32 [?,%E],[?,%J]
  %r = load i32, ptr %p
  ret i32 %r
```

ϕ locations

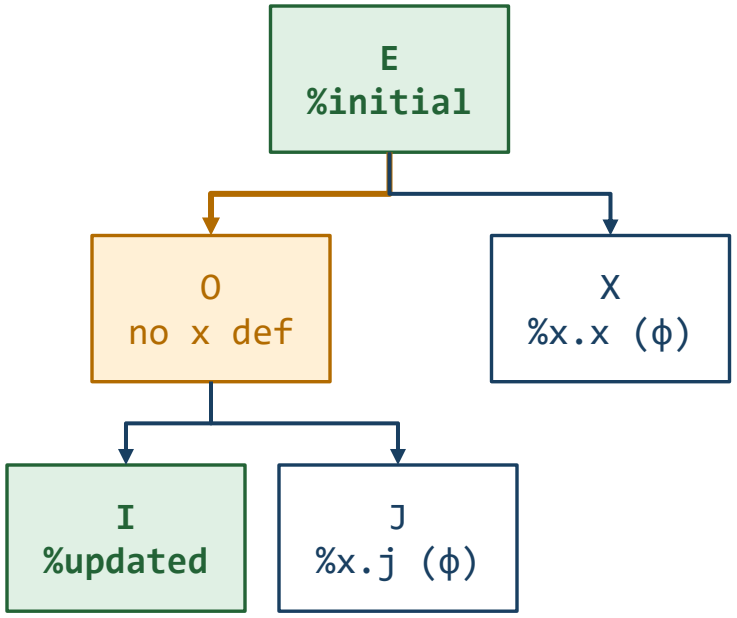


- Stores define x in E and I.
- DF(E) = ∅, DF(I) = {J}.
- The ϕ in J is a new definition.
- DF(J) = {X}.

Nested 2: Fill J's Incoming Values

```
E:
  %p = alloca i32
  %initial = add i32 %a, 1
  store i32 %initial, ptr %p
  br i1 %c, label %O, label %X
O:
  br i1 %d, label %I, label %J
I:
  %updated = mul i32 %b, 2
  store i32 %updated, ptr %p
  br label %J
J:
  %x.j = phi i32 [%initial,%O],[%updated,%I]
  br label %X
X:
  %x.x = phi i32 [,%E],[,%J]
  %r = load i32, ptr %p
  ret i32 %r
```

Start at predecessors O and I

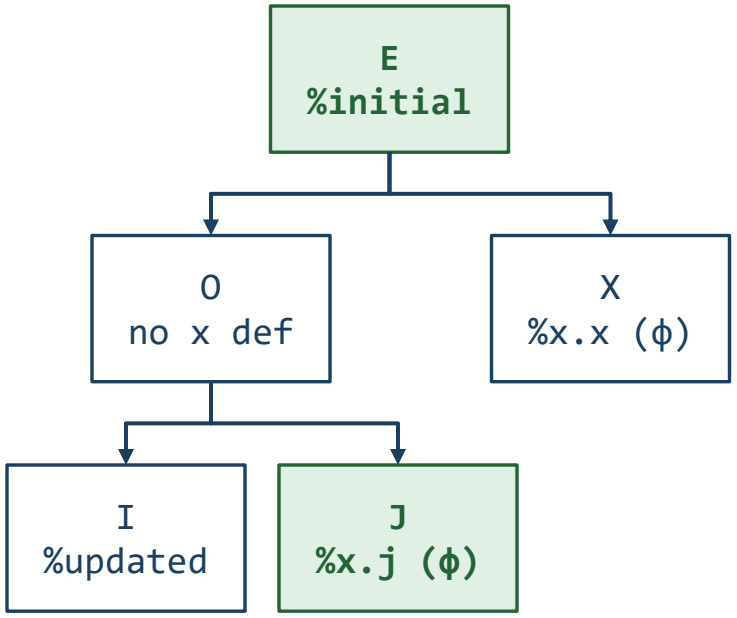


O has no local x definition.
E is its nearest defining ancestor.
I supplies its local %updated.

Nested 3: Fill X's Incoming Values

```
E:
  %p = alloca i32
  %initial = add i32 %a, 1
  store i32 %initial, ptr %p
  br i1 %c, label %O, label %X
O:
  br i1 %d, label %I, label %J
I:
  %updated = mul i32 %b, 2
  store i32 %updated, ptr %p
  br label %J
J:
  %x.j = phi i32 [%initial,%O],[%updated,%I]
  br label %X
X:
  %x.x = phi i32 [%initial,%E],[%x.j,%J]
  %r = load i32, ptr %p
  ret i32 %r
```

Start at predecessors E and J



E → X uses %initial.

J → X uses its local ϕ, %x.j.

I's definition does not dominate J.

Nested 4: Replace the Load

```
E:
  %p = alloca i32
  %initial = add i32 %a, 1
  store i32 %initial, ptr %p
  br i1 %c, label %O, label %X
O:
  br i1 %d, label %I, label %J
I:
  %updated = mul i32 %b, 2
  store i32 %updated, ptr %p
  br label %J
J:
  %x.j = phi i32 [%initial,%O],[%updated,%I]
  br label %X
X:
  %x.x = phi i32 [%initial,%E],[%x.j,%J]
  %r = load i32, ptr %p
  ret i32 %x.x
```

Before, in X

```
%r = load i32, ptr %p
ret i32 %r
```

After, in X

```
ret i32 %x.x
```

X's own ϕ is the nearest definition before the load.

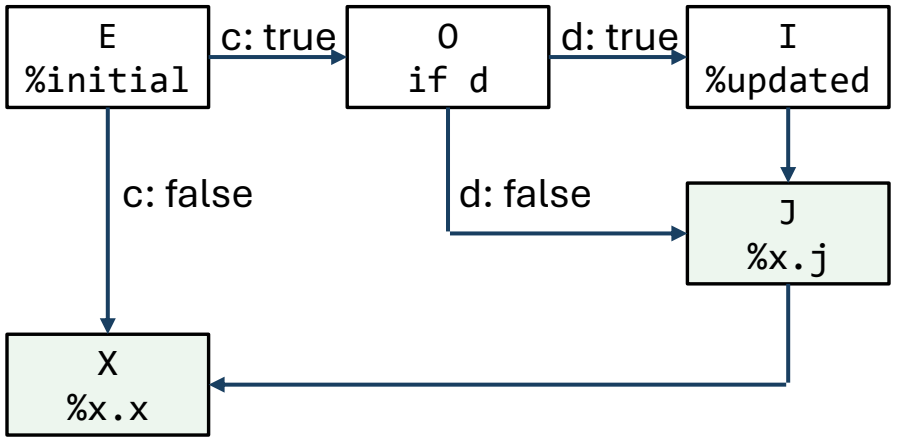
Nested 5: Delete the Stores

```
E:
  %p = alloca i32
  %initial = add i32 %a, 1
  store i32 %initial, ptr %p
  br i1 %c, label %O, label %X
O:
  br i1 %d, label %I, label %J
I:
  %updated = mul i32 %b, 2
  store i32 %updated, ptr %p
  br label %J
J:
  %x.j = phi i32 [%initial,%O],[%updated,%I]
  br label %X
X:
  %x.x = phi i32 [%initial,%E],[%x.j,%J]
  ret i32 %x.x
```

%initial and %updated remain as ϕ operands.

Nested 6: Delete the Alloca

```
E:
  %p = alloca i32
  %initial = add i32 %a, 1
  br i1 %c, label %O, label %X
O:
  br i1 %d, label %I, label %J
I:
  %updated = mul i32 %b, 2
  br label %J
J:
  %x.j = phi i32 [%initial,%O],[%updated,%I]
  br label %X
X:
  %x.x = phi i32 [%initial,%E],[%x.j,%J]
  ret i32 %x.x
```



Loop: Initial IR

```
pre:
  %p = alloca i32
  store i32 0, ptr %p
  br label %header
header:
  %cur = load i32, ptr %p
  %c = icmp slt i32 %cur, %n
  br i1 %c, label %latch, label %exit
latch:
  %old = load i32, ptr %p
  %next = add i32 %old, 1
  store i32 %next, ptr %p
  br label %header
exit:
  %r = load i32, ptr %p
  ret i32 %r
```

RustY

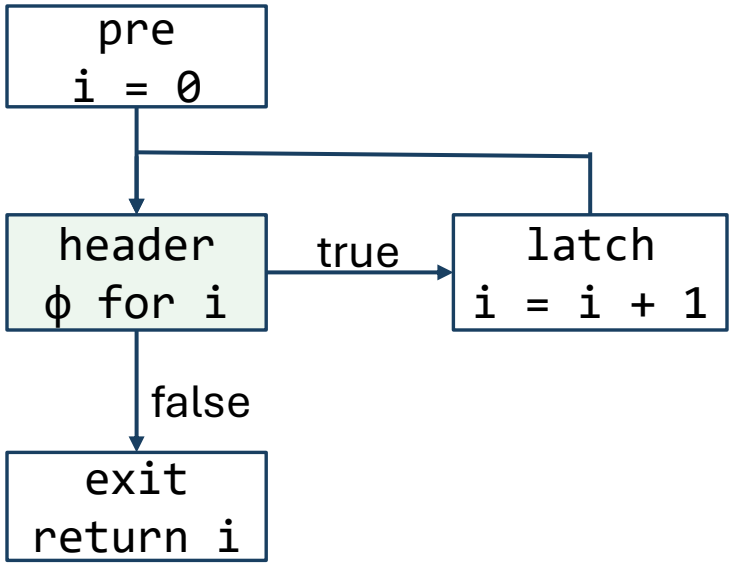
```
fn count(n: i32) -> i32 {
  let mut i: i32 = 0;
  while i < n {
    i = i + 1;
  }
  return i;
}
```

Promote the slot for i.

Loop 1: Mark the ϕ Location

```
pre:
  %p = alloca i32
  store i32 0, ptr %p
  br label %header
header:
  %cur = load i32, ptr %p
  %c = icmp slt i32 %cur, %n
  br i1 %c, label %latch, label %exit
latch:
  %old = load i32, ptr %p
  %next = add i32 %old, 1
  store i32 %next, ptr %p
  br label %header
exit:
  %r = load i32, ptr %p
  ret i32 %r
```

ϕ location



The initial edge and backedge meet at header.

Loop 2: Fill In the Incoming Values

```
pre:
  %p = alloca i32
  store i32 0, ptr %p
  br label %header
header:
  %i = phi i32 [0,%pre],[%next,%latch]
  %cur = load i32, ptr %p
  %c = icmp slt i32 %cur, %n
  br i1 %c, label %latch, label %exit
latch:
  %old = load i32, ptr %p
  %next = add i32 %old, 1
  store i32 %next, ptr %p
  br label %header
exit:
  %r = load i32, ptr %p
  ret i32 %r
```

Header ϕ inputs

```
%i = phi i32
  [0,%pre],
  [%next,%latch]
```

pre supplies 0.

The backedge supplies %next.

Loop 3: Replace All Three Loads

```
pre:
  %p = alloca i32
  store i32 0, ptr %p
  br label %header
header:
  %i = phi i32 [0,%pre],[%next,%latch]
  %c = icmp slt i32 %i, %n
  br i1 %c, label %latch, label %exit
latch:
  %next = add i32 %i, 1
  store i32 %next, ptr %p
  br label %header
exit:
  ret i32 %i
```

Updated instructions

```
%c = icmp slt i32 %i, %n
%next = add i32 %i, 1
ret i32 %i
```

The comparison, increment, and return now use %i.

There are no loads =>
No need to store

Loop 4: Delete the Stores

```
pre:
  %p = alloca i32
  br label %header
header:
  %i = phi i32 [0,%pre],[%next,%latch]
  %c = icmp slt i32 %i, %n
  br i1 %c, label %latch, label %exit
latch:
  %next = add i32 %i, 1
  br label %header
exit:
  ret i32 %i
```

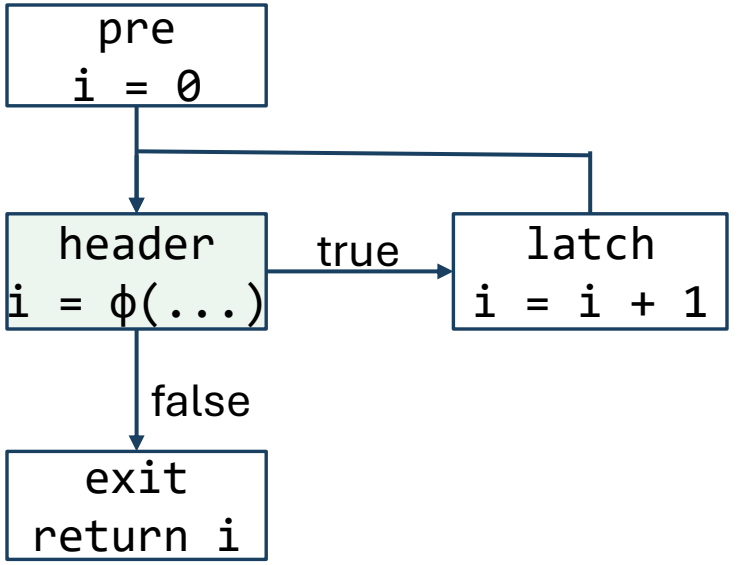
Removed instructions

```
store i32 0, ptr %p
store i32 %next, ptr %p
```

%next remains: the ϕ uses it on the backedge.

Loop 5: Delete the Alloca

```
pre:
  br label %header
header:
  %i = phi i32 [0,%pre],[%next,%latch]
  %c = icmp slt i32 %i, %n
  br i1 %c, label %latch, label %exit
latch:
  %next = add i32 %i, 1
  br label %header
exit:
  ret i32 %i
```



Removed: %p = alloca i32

The loop uses a ϕ, arithmetic, and branches. No slot remains.

Alive2: Checking a Load Replacement

Source

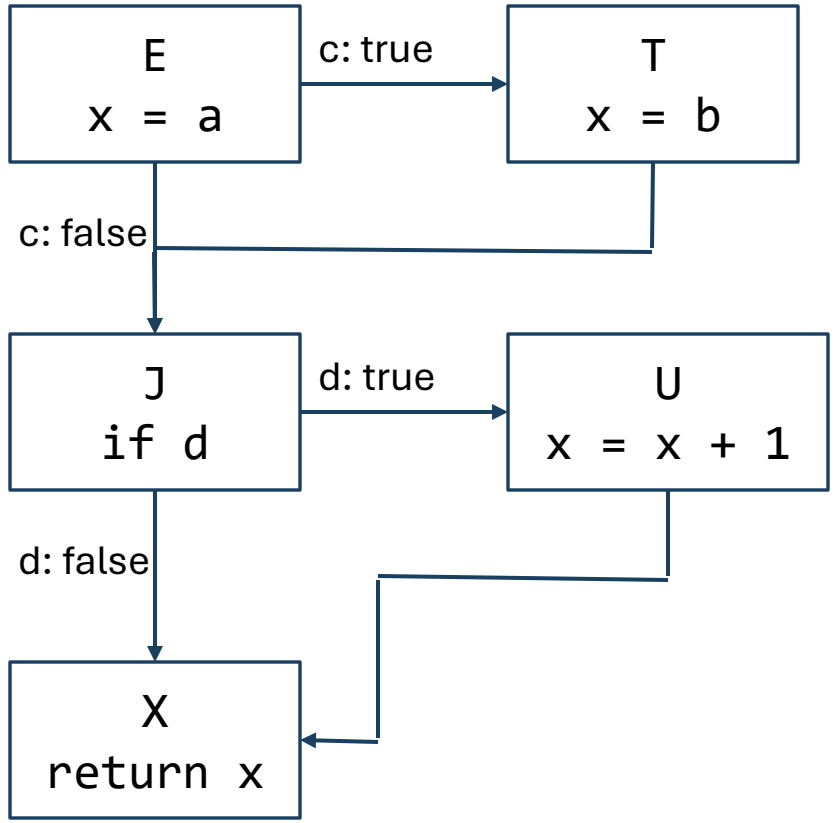
```
define i32 @src(i32 %a) {
  %p = alloca i32
  store i32 %a, ptr %p
  %old = load i32, ptr %p
  %next = add i32 %old, 1
  store i32 %next, ptr %p
  %r = load i32, ptr %p
  ret i32 %r
}
```

Target

```
define i32 @tgt(i32 %a) {
  %next = add i32 %a, 1
  ret i32 %next
}
```

Exercise: Two Consecutive Conditionals

```
fn update(a: i32, b: i32,
          c: bool, d: bool) -> i32 {
  let mut x: i32 = a;
  if c { x = b; }
  if d { x = x + 1; }
  return x;
}
```



Place ϕs, then replace every load with the correct SSA value.

Exercise Answer

```
E:
  br i1 %c, label %T, label %J
T:
  br label %J
J:
  %x.j = phi i32 [%a,%E],[%b,%T]
  br i1 %d, label %U, label %X
U:
  %next = add i32 %x.j, 1
  br label %X
X:
  %x.x = phi i32 [%x.j,%J],[%next,%U]
  ret i32 %x.x
```

D = {E, T, U}

DF(T) = {J}
DF(U) = {X}
DF(J) = ∅

The ϕ result in J feeds both the add and the ϕ in X.

Appendix: Running mem2reg in RustY

Run these commands from compiler/ in the course Linux environment.

Before promotion

```
dune exec rustyc -- --emit-llvm input.rs -o before.ll
```

After promotion

```
dune exec rustyc -- --emit-llvm --mem2reg input.rs -o after.ll
```

Compare allocas, loads, stores, and ϕ instructions.